



Rapport de Projet IA

Quoridor — Minimax Alpha-Bêta

Membre du projet :
Danil GUIDJOU
Anis HAMMOUCHE

Table des matières

1	Présentation générale du jeu	2
1.1	Règles principales	2
1.2	Conditions de victoire	2
1.3	Contrainte fondamentale	2
1.4	Simplifications retenues	2
2	Implémentation et validation du moteur de jeu	2
2.1	Représentation de l'état	2
2.2	Génération des coups légaux	3
2.3	Validation des murs	3
2.4	Détection des états terminaux	3
2.5	Tests unitaires	3
3	Algorithmes de décision et fonctions d'évaluation	3
3.1	Algorithme Minimax	3
3.2	Élagage alpha-bêta	3
3.3	Optimisation du facteur de branchement	4
3.4	Trois fonctions d'évaluation	4
3.5	Analyse expérimentale	5
3.6	Ordonnancement des coups et élagage	5
4	Tournoi entre les intelligences artificielles	5
4.1	Organisation générale	5
4.2	Déroulement du tournoi	6
4.3	Tournoi Minimax pur	7
4.4	Tournoi avec élagage alpha-bêta	13
4.5	Comparaison Minimax pur vs Alpha-Bêta	15
5	Utilisation de l'IA générative	16
5.1	Outils utilisés	16
5.2	Ce qu'on en a fait	16
5.3	Ce qui a marché et ce qui n'a pas marché	16
6	Difficultés rencontrées	17
6.1	Auto-blocage de l'IA	17
6.2	Oscillation entre positions	17
6.3	Temps de calcul à profondeur 3	17
6.4	Gestion des sauts	17
6.5	Pose de murs lors d'un match entre deux IA	17

1 Présentation générale du jeu

Le Quoridor est un jeu de plateau créé par Mirko Marchesi. Deux joueurs s'affrontent sur une grille de 9×9 cases. Chacun contrôle un pion et dispose de 10 murs à poser au fil de la partie. Le joueur 1 démarre au centre de la première ligne (position $(4, 0)$), le joueur 2 au centre de la dernière (position $(4, 8)$).

1.1 Règles principales

À chaque tour, un joueur choisit entre deux actions :

- Déplacer son pion d'une case (haut, bas, gauche ou droite), à condition qu'un mur ne bloque pas le passage.
- Poser un mur de 2 cases de long, horizontal ou vertical, ce qui consomme un mur de son stock.

Quand l'adversaire occupe la case adjacente, on peut le sauter (avancer de 2 cases). Si un mur ou le bord empêche ce saut direct, on saute en diagonale.

1.2 Conditions de victoire

- Le joueur 1 gagne en atteignant la ligne $y = 8$ (bord opposé).
- Le joueur 2 gagne en atteignant la ligne $y = 0$ (bord opposé).

1.3 Contrainte fondamentale

On ne peut pas poser un mur s'il coupe tout chemin vers la ligne d'arrivée d'un des deux joueurs. On vérifie ça avec un BFS (parcours en largeur) après chaque tentative de pose.

1.4 Simplifications retenues

Les règles sont identiques à la version officielle. On a juste ajouté une limite de 200 coups par partie, pour éviter que les matchs entre IA tournent en boucle.

2 Implémentation et validation du moteur de jeu

2.1 Représentation de l'état

Toute la logique du jeu est dans la classe `QuoridorBoard`. L'état d'une partie tient dans quatre attributs :

- `positions` : dictionnaire $\{1 : (x_1, y_1), 2 : (x_2, y_2)\}$ pour la position de chaque pion.
- `walls` : ensemble de tuples $(x, y, orientation)$ avec $orientation \in \{'H', 'V'\}$. Un mur horizontal en (x, y) bloque le passage entre les lignes y et $y + 1$ sur les colonnes x et $x + 1$.
- `walls_count` : dictionnaire $\{1 : n_1, 2 : n_2\}$, le nombre de murs restants par joueur.
- `winner` : `None` tant que la partie continue, sinon l'identifiant du vainqueur.

2.2 Génération des coups légaux

`get_legal_pawn_moves(player_id)` retourne la liste des cases accessibles. La méthode gère les murs, les bords du plateau et les sauts (directs ou diagonaux quand l'adversaire est adjacent). Elle s'appuie sur `get_accessible_neighbors(x, y)` qui donne les voisins non bloqués par un mur.

2.3 Validation des murs

`place_wall` passe par plusieurs vérifications :

1. Le joueur a-t-il encore des murs ? (`stock > 0`)
2. Les coordonnées sont-elles dans la grille ? ($0 \leq x, y \leq 7$)
3. Pas de chevauchement avec un mur existant ? (`_is_wall_placement_valid`)
4. Les deux joueurs ont-ils toujours un chemin ? (BFS via `is_path_available`)

2.4 Détection des états terminaux

La victoire est vérifiée dans `move_pawn` : dès qu'un pion atteint sa ligne cible, `winner` est mis à jour et la partie s'arrête.

2.5 Tests unitaires

15 tests avec `pytest` couvrent l'initialisation, la copie du board, les déplacements (cas normal, hors plateau, bloqué par mur), les sauts (direct, diagonal, au bord), la pose de murs (valide, stock épuisé, chevauchements, blocage) et la détection de victoire. Ils passent tous.

3 Algorithmes de décision et fonctions d'évaluation

3.1 Algorithme Minimax

On a d'abord implémenté le Minimax classique, sans aucune optimisation de recherche (`src/ia/minimax.py`). L'idée est simple : on explore l'arbre de jeu jusqu'à une profondeur fixée. À chaque nœud, le joueur courant (MAX) choisit le coup qui maximise son score, et l'adversaire (MIN) choisit celui qui le minimise. À la profondeur limite, on évalue la position avec une heuristique.

L'algorithme explore tous les nœuds sans exception, ce qui donne une complexité en $O(b^d)$ avec b le facteur de branchement et d la profondeur.

`get_best_move` appelle le minimax sur chaque coup possible et garde le meilleur. On a aussi ajouté un mécanisme anti-oscillation : l'IA garde en mémoire ses 6 dernières positions, et pénalise de -50 tout retour sur une case récente. Sans ça, les IA D1 font des aller-retours sans fin.

3.2 Élagage alpha-bêta

Le Minimax pur explore beaucoup de branches inutiles. Si on sait déjà qu'un coup donne un score de 10, pourquoi continuer à explorer une branche où l'adversaire peut nous forcer en dessous ? C'est le principe de l'élagage alpha-bêta (`src/ia/elagage.py`).

On maintient deux bornes : α (le meilleur score garanti pour MAX) et β (le meilleur score garanti pour MIN). Dès que $\alpha \geq \beta$, on coupe la branche. Dans le cas optimal, ça réduit la complexité de $O(b^d)$ à $O(b^{d/2})$. En pratique, ça veut dire que l'élagage à profondeur 4 explore à peu près autant de nœuds que le Minimax pur à profondeur 2.

L' α de la racine est mis à jour avec le score brut, pas le score pénalisé par l'anti-oscillation. Sinon l'élagage serait faussé.

3.3 Optimisation du facteur de branchement

Le Quoridor a un facteur de branchement d'environ 72 (4-5 déplacements + ~67 murs possibles). Trop pour chercher en profondeur, même avec élagage. Le module `moves_optimization.py` ramène ça à 10–15 coups :

1. Les déplacements de pion en premier, triés par progression vers l'objectif.
2. Pour les murs, seuls ceux dans une zone autour des deux joueurs (marge de 2 cases) sont considérés.
3. Chaque mur candidat est simulé via BFS : on ne garde que ceux qui gênent plus l'adversaire que nous ($gain_{adv} \geq cost_{self}$ et $gain_{adv} > 0$).
4. Les murs restants sont triés par gain net décroissant, pour que l'élagage coupe vite.

Mertens (2006) était limité à la profondeur 2 avec un facteur de branchement de 60–70. Notre filtrage permet d'atteindre D3 et D4.

3.4 Trois fonctions d'évaluation

Heuristique simple (`heuristic_simple_distance`)

$$score = (dist_{adv} - dist_{joueur}) \times 10 \quad (1)$$

On compare la distance BFS de chaque joueur vers sa ligne d'arrivée. Plus l'adversaire est loin et plus on est proche, mieux c'est.

Heuristique avancée (`heuristic_shortest_path`)

$$score = (dist_{adv} - dist_{joueur}) \times 10 + (murs_{joueur} - murs_{adv}) \times 5 \quad (2)$$

Même base, mais on valorise aussi le fait d'avoir gardé ses murs. Un joueur qui a des murs en réserve peut encore bloquer l'adversaire.

Heuristique experte (`heuristic_expert`)

$$score = (dist_{adv} - dist_{joueur}) \times 10 + \Delta_{murs} \times 5 + progression \times 3 + bonus_{tempo} + bonus_{endgame} \quad (3)$$

- *progression* : combien le pion a avancé vers son objectif (y pour J1, $8 - y$ pour J2). Ça pousse à avancer au lieu de stagner.
- *bonus_{tempo}* : quand on mène la course et qu'on a des murs pour se défendre, le score est amplifié ($+\Delta_{dist} \times 3$).
- *bonus_{endgame}* : si l'adversaire n'a plus de murs, il ne peut pas nous bloquer donc on essaye d'aller tout droit.

3.5 Analyse expérimentale

Le tableau 1 résume les performances en poules (9 matchs par IA).

TABLE 1 – Performances en poules par configuration (sur 9 matchs par IA)

IA	Config	Victoires	Défaites	Temps moy. (s)
Naïve	D1, simple	2	7	0.3
Agressive	D1, expert	3	6	0.2
Prudente	D1, advanced	2	7	0.2
Calculatrice	D2, simple	4	5	2.3
Stratège	D2, advanced	4	5	2.1
Commandante	D2, expert	5	4	1.9
Visionnaire	D3, advanced	6	3	18.5
Maîtresse	D3, expert	7	2	19.2

La profondeur compte le plus. Les IA D3 gagnent la majorité de leurs matchs. À profondeur égale, expert bat advanced, qui bat simple.

Le temps monte vite : 0.2s par coup à D1, 2s à D2, presque 20s à D3. Chercher plus loin donne de meilleures décisions, mais ça se paye.

3.6 Ordonnancement des coups et élagage

L'ordre dans lequel on explore les coups a un vrai impact sur l'élagage. En testant les déplacements d'abord (triés par progression) puis les murs par gain décroissant, on tombe vite sur de bons coups. La fenêtre $[\alpha, \beta]$ se resserre et on coupe plus de branches.

On a aussi vu que l'heuristique experte aide l'élagage à couper plus tôt, parce qu'elle différencie mieux les positions. C'est probablement pour ça que la Commandante (D2, expert) bat la Visionnaire (D3, advanced) malgré une profondeur inférieure : elle explore mieux avec moins.

4 Tournoi entre les intelligences artificielles

4.1 Organisation générale

On a mis en place deux tournois distincts avec la même structure mais des algorithmes différents :

- Tournoi Minimax pur : sans élagage (`src/ia/minimax.py`)
- Tournoi Alpha-Bêta : avec élagage alpha-bêta (`src/ia/elagage.py`)

Chaque tournoi oppose 12 IA, toutes les combinaisons de profondeur (D1 à D4) et d'heuristique (simple, advanced, expert) :

TABLE 2 – Les 12 instances du tournoi

Nom	Profondeur	Stratégie
D1-Simple	1	simple
D1-Advanced	1	advanced
D1-Expert	1	expert
D2-Simple	2	simple
D2-Advanced	2	advanced
D2-Expert	2	expert
D3-Simple	3	simple
D3-Advanced	3	advanced
D3-Expert	3	expert
D4-Simple	4	simple
D4-Advanced	4	advanced
D4-Expert	4	expert

4.2 Déroulement du tournoi

Le tournoi se déroule en deux phases.

Phase de poules

Les 12 IA sont réparties dans 3 poules de 4, en mélangeant les profondeurs pour éviter de regrouper toutes les IA fortes :

- Poule A : D1-Simple, D2-Expert, D3-Advanced, D4-Simple
- Poule B : D1-Advanced, D2-Simple, D3-Expert, D4-Advanced
- Poule C : D1-Expert, D2-Advanced, D3-Simple, D4-Expert

Chaque paire joue 3 matchs avec alternance J1/J2. Ça donne $6 \text{ paires} \times 3 \text{ matchs} = 18$ matchs par poule, soit 54 matchs en poules.

L'alternance J1/J2 compte parce que le premier joueur a un léger avantage. Trois matchs par paire permettent de le compenser.

Le classement se fait au nombre de victoires. En cas d'égalité, l'ordre de déclaration est conservé.

Phase éliminatoire

Les 8 meilleurs passent en élimination directe :

- Les 2 premiers de chaque poule (6 qualifiés directs)
- Les 2 meilleurs 3^{èmes}

La suite se joue en matchs aller-retour (pour alterner le joueur qui commence la partie) :

1. Quarts de finale : 4 matchs avec croisements inter-poules
2. Demi-finales : les 4 vainqueurs
3. Petite finale : les 2 perdants des demies pour la 3^{ème} place
4. Grande finale : les 2 vainqueurs pour le titre

Si c'est 1-1, on départage par :

1. Le nombre de coups du match gagné (le plus court l'emporte)
2. Le nombre de murs restants du vainqueur
3. Un 3^{ème} match avec ordre J1/J2 tiré au sort

Au total, la phase éliminatoire fait environ 16 matchs, pour un total d'environ 70 matchs par tournoi.

4.3 Tournoi Minimax pur

Ce tournoi utilise le Minimax pur. Chaque nœud de l'arbre est exploré sans aucune coupure. Complexité : $O(b^d)$ avec $b \approx 12$ après filtrage.

Pour éviter des matchs redondants, on a implémenté un tie-breaking : quand deux coups ont le même score, il y a 20% de chance de changer de coup. On l'a ensuite retiré car ça introduisait des incohérences où une IA de profondeur 4 perdait face à une IA de profondeur 1.

Durée du tournoi : environ 2 heures, à cause des IA D4 qui explorent l'arbre en entier.

Résultats

On a exécuté le tournoi deux fois : une première fois avec tie-breaking (20% d'aléatoire), puis une seconde fois sans tie-breaking. Le tableau 3 compare les deux classements côte à côte.

TABLE 3 – Classement Minimax pur : avec vs sans tie-breaking

Rang	Avec tie-breaking			Sans tie-breaking		
	IA	D	Strat.	IA	D	Strat.
1	D2-Expert	2	expert	D4-Advanced	4	advanced
2	D4-Expert	4	expert	D2-Expert	2	expert
3	D3-Expert	3	expert	D2-Simple	2	simple
4	D2-Advanced	2	advanced	D4-Simple	4	simple
5	D4-Simple	4	simple	D3-Expert	3	expert
6	D4-Advanced	4	advanced	D4-Expert	4	expert
7	D3-Simple	3	simple	D2-Advanced	2	advanced
8	D3-Advanced	3	advanced	D3-Advanced	3	advanced
9	D2-Simple	2	simple	D3-Simple	3	simple
10	D1-Simple	1	simple	D1-Simple	1	simple
11	D1-Advanced	1	advanced	D1-Advanced	1	advanced
12	D1-Expert	1	expert	D1-Expert	1	expert

Analyse des résultats

Avec le tie-breaking, l'heuristique expert occupe le podium en entier (D2, D4, D3). Mais D2-Expert finit premier, devant D4-Expert. C'est une anomalie : une IA de profondeur 2 ne devrait pas battre une de profondeur 4 avec la même heuristique. Le tie-breaking en est la cause — à D4, l'IA évalue plus de coups à la racine et tombe plus souvent sur des scores égaux, ce qui augmente les chances de dévier du meilleur coup.

Sans le tie-breaking, le classement change beaucoup. D4-Advanced passe en tête, et les résultats sont plus variés en termes d'heuristiques. On remarque que D2-Expert et D2-Simple montent au podium, ce qui montre qu'à profondeur 2 l'algorithme est assez stable pour bien

jouer sans aléatoire. D4 ne domine pas autant qu'on pourrait l'espérer : D4-Advanced est premier mais D4-Expert n'est que 6^{ème} et D4-Simple 4^{ème}.

Dans les deux cas, les IA D1 finissent en bas du classement (rangs 10–12). La profondeur 1 ne suffit pas, avec ou sans tie-breaking.

Graphiques d'analyse

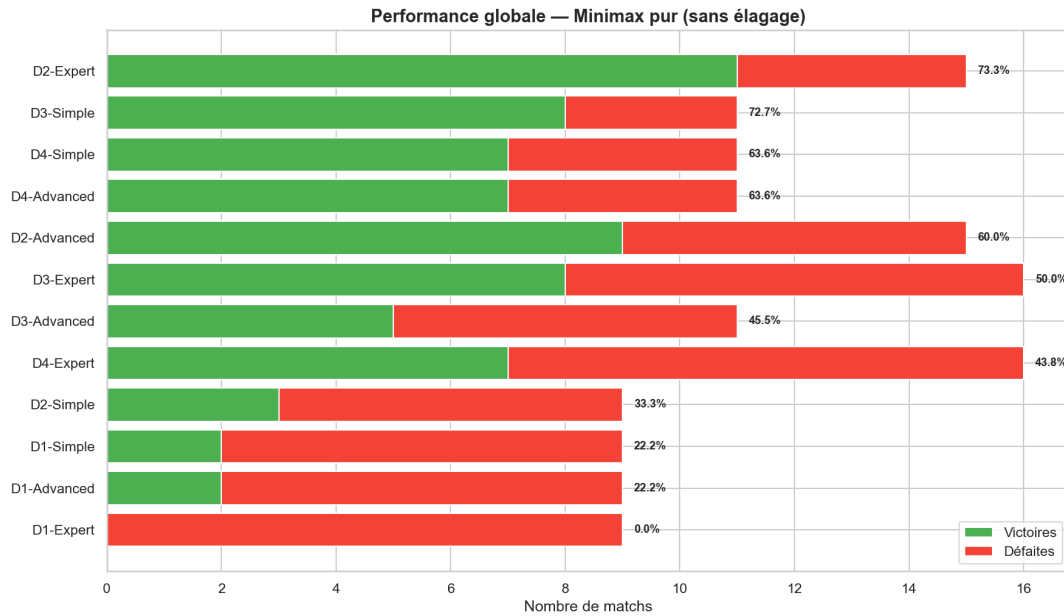


FIGURE 1 – Victoires et défaites — Minimax pur (avec tie-breaking)

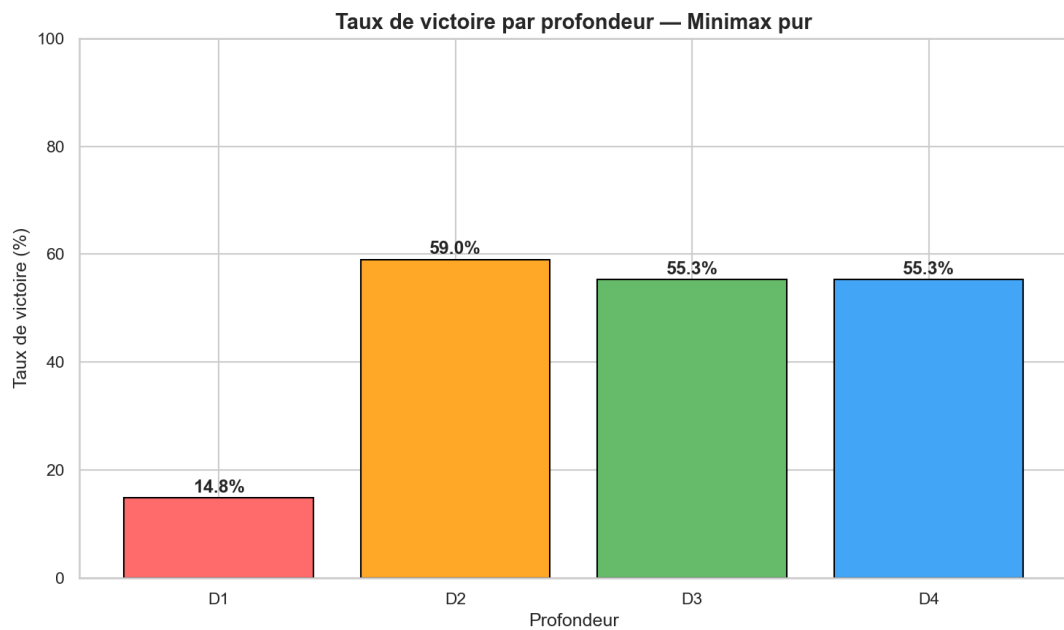


FIGURE 2 – Taux de victoire par profondeur — Minimax pur (avec tie-breaking)

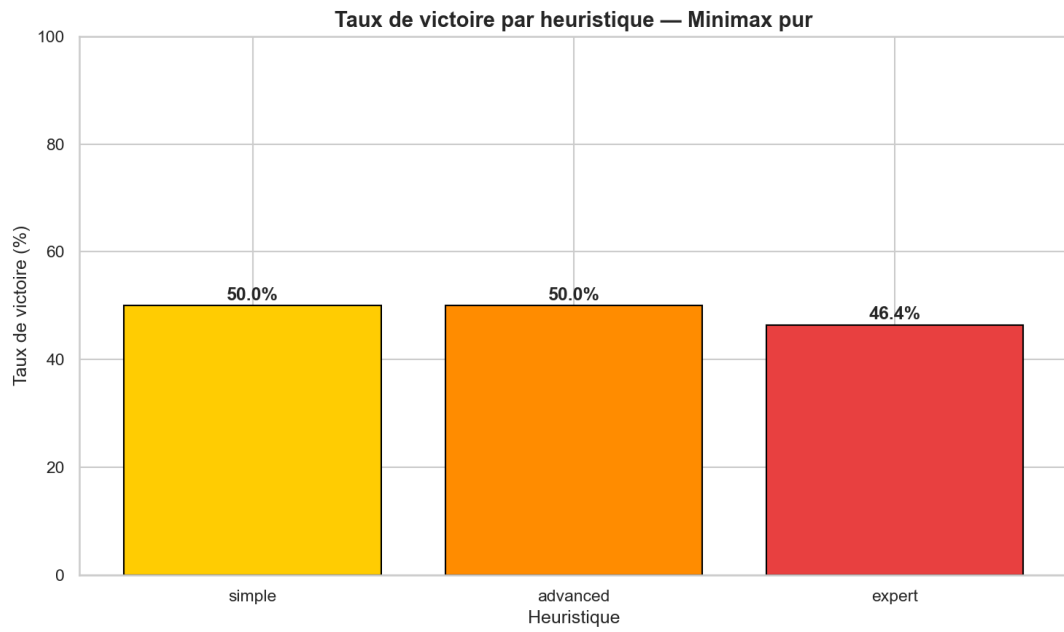


FIGURE 3 – Taux de victoire par heuristique — Minimax pur (avec tie-breaking)

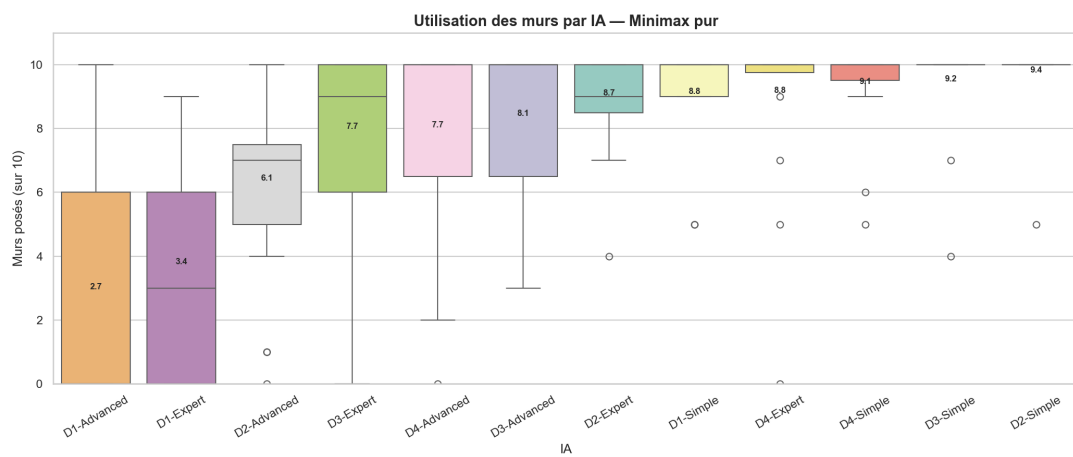


FIGURE 4 – Utilisation des murs — Minimax pur (avec tie-breaking)

Graphiques sans tie-breaking

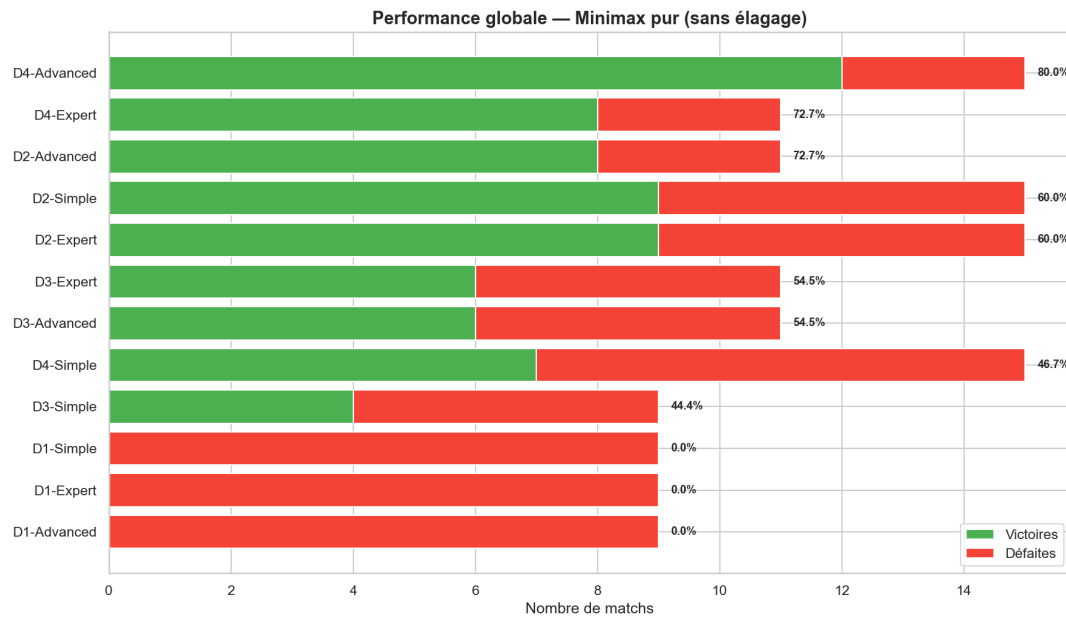


FIGURE 5 – Victoires et défaites — Minimax pur (sans tie-breaking)

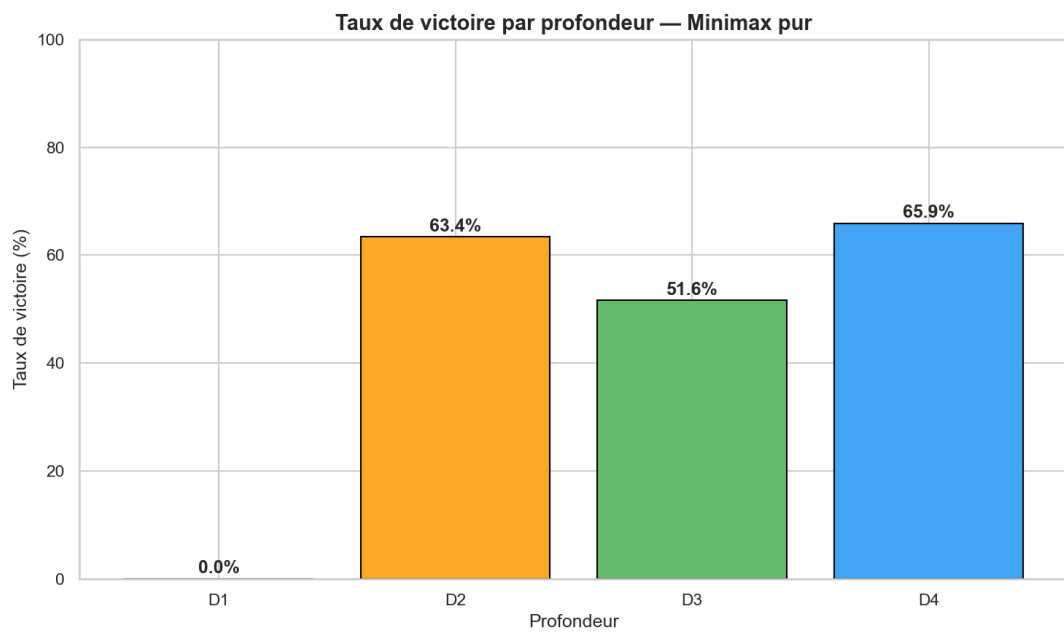


FIGURE 6 – Taux de victoire par profondeur — Minimax pur (sans tie-breaking)

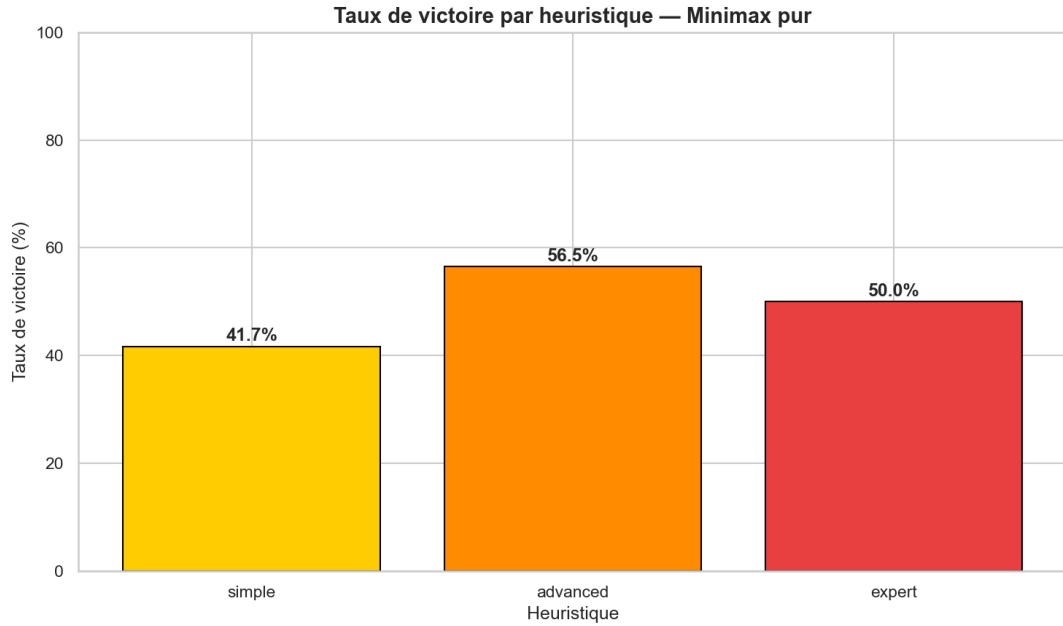


FIGURE 7 – Taux de victoire par heuristique — Minimax pur (sans tie-breaking)

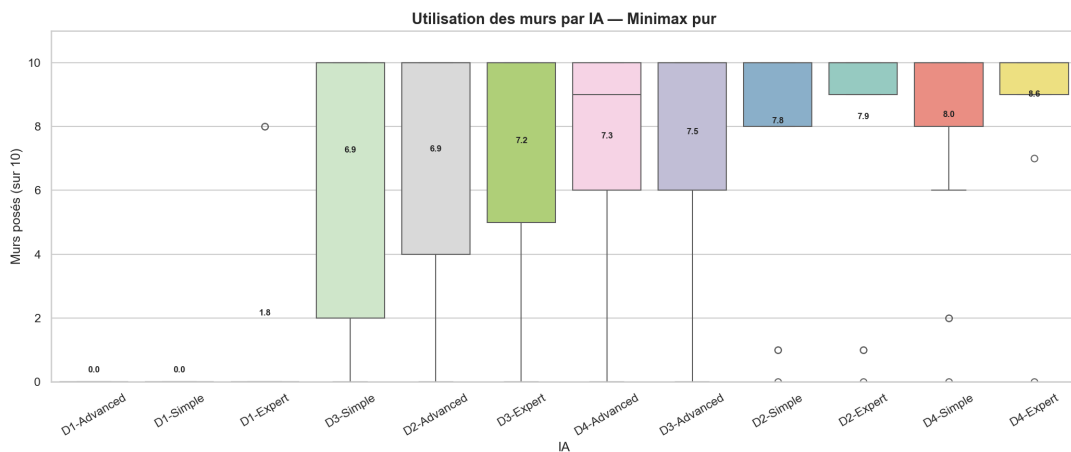


FIGURE 8 – Utilisation des murs — Minimax pur (sans tie-breaking)

4.4 Tournoi avec élagage alpha-bêta

Même structure, mais l'algorithme utilise l'élagage alpha-bêta (`src/ia/elagage.py`). Les branches inutiles sont coupées, ce qui réduit la complexité de $O(b^d)$ à $O(b^{d/2})$ dans le cas optimal. En pratique, l'élagage à D4 explore à peu près autant de nœuds que le Minimax pur à D2.

Ce tournoi n'utilise aucun tie-breaking aléatoire : à score égal, le premier coup trouvé est conservé. Les résultats sont donc reproductibles.

Durée du tournoi : environ 20 minutes. Beaucoup plus rapide grâce aux coupures.

Résultats

TABLE 4 – Classement final — Alpha-Bêta

Rang	IA	Profondeur	Stratégie
1	D4-Advanced	4	advanced
2	D2-Advanced	2	advanced
3	D3-Advanced	3	advanced
4	D4-Simple	4	simple
5	D2-Simple	2	simple
6	D4-Expert	4	expert
7	D3-Expert	3	expert
8	D2-Expert	2	expert
9	D3-Simple	3	simple
10	D1-Simple	1	simple
11	D1-Advanced	1	advanced
12	D1-Expert	1	expert

Analyse des résultats

Cette fois, c'est l'heuristique advanced qui occupe le podium (D4, D2, D3). On ne s'y attendait pas : l'expert, avec ses 5 composantes, devrait en théorie évaluer mieux. Mais l'élagage change la donne.

L'explication : l'heuristique expert produit des scores plus dispersés (5 termes additifs), ce qui rend les coupures moins prévisibles. L'élagage peut couper une branche qui, avec des scores plus homogènes, aurait été explorée et aurait mené au meilleur coup. L'advanced, avec seulement 2 termes (distance + murs), produit des évaluations plus stables et guide l'élagage de manière plus fiable.

La profondeur reste le facteur principal. D4-Advanced bat D3-Advanced, D4-Simple bat D2-Simple. À heuristique égale, chercher plus loin donne toujours un avantage.

D2-Advanced qui atteint la finale est un résultat qui mérite attention. Une IA de profondeur 2 qui rivalise avec D4, c'est possible parce que l'élagage rend D2 très rapide (~ 1 s/coup) et l'heuristique advanced guide bien les coupures.

Deux matchs se sont terminés en nul (200 coups) en demi-finale, entre des IA de niveau très proche. L'anti-oscillation empêche les boucles courtes (6 dernières positions), mais des cycles plus longs passent à travers.

Les IA D1 sont toutes éliminées en poules (rangs 10–12), comme dans le tournoi Minimax.

Graphiques d'analyse

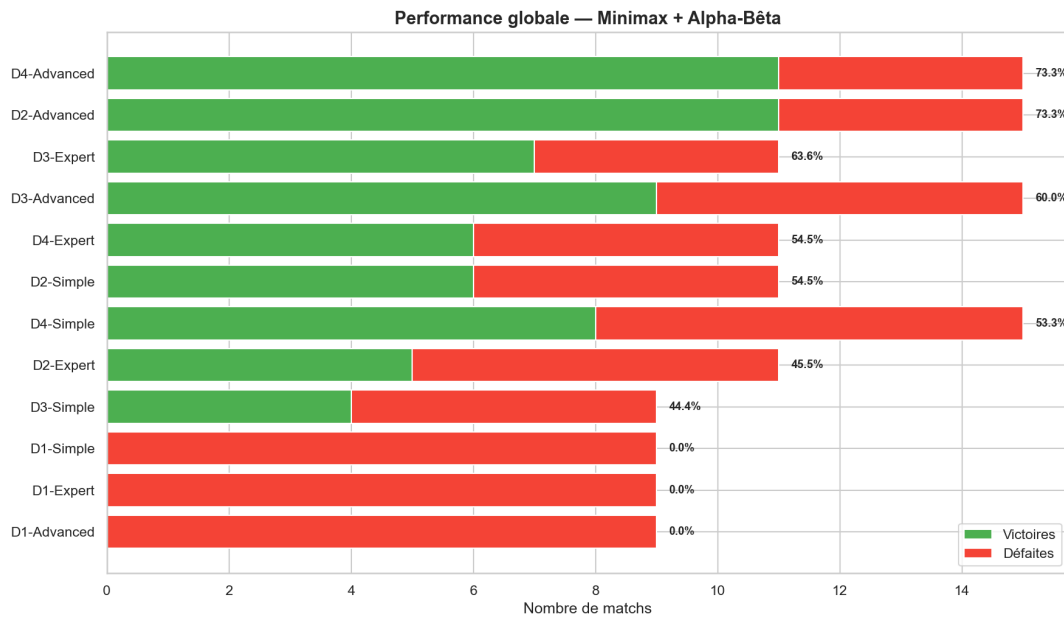


FIGURE 9 – Victoires et défaites — Alpha-Bêta

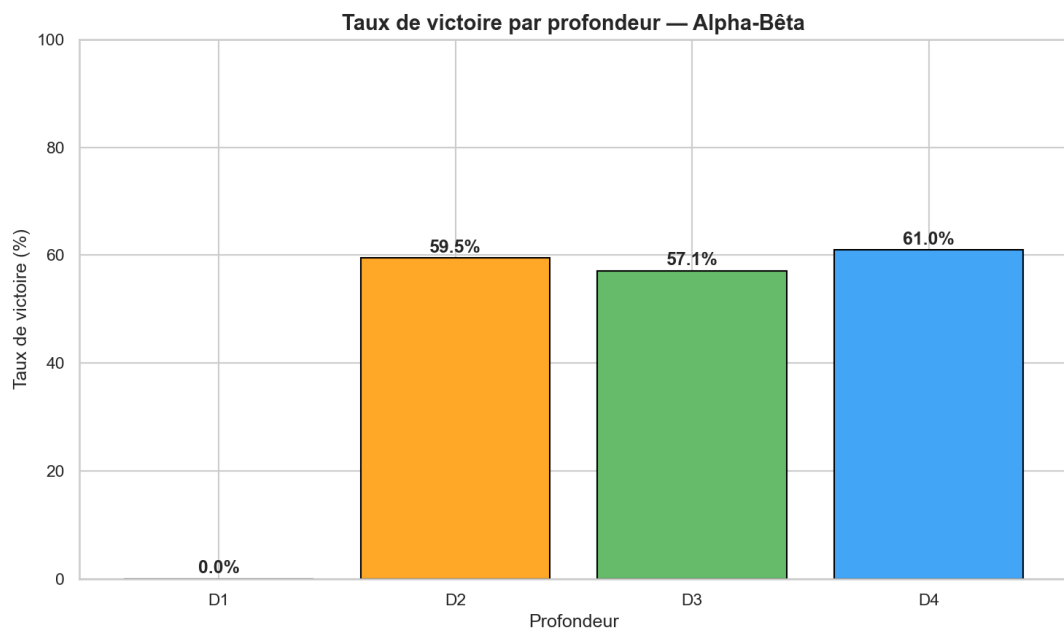


FIGURE 10 – Taux de victoire par profondeur — Alpha-Bêta

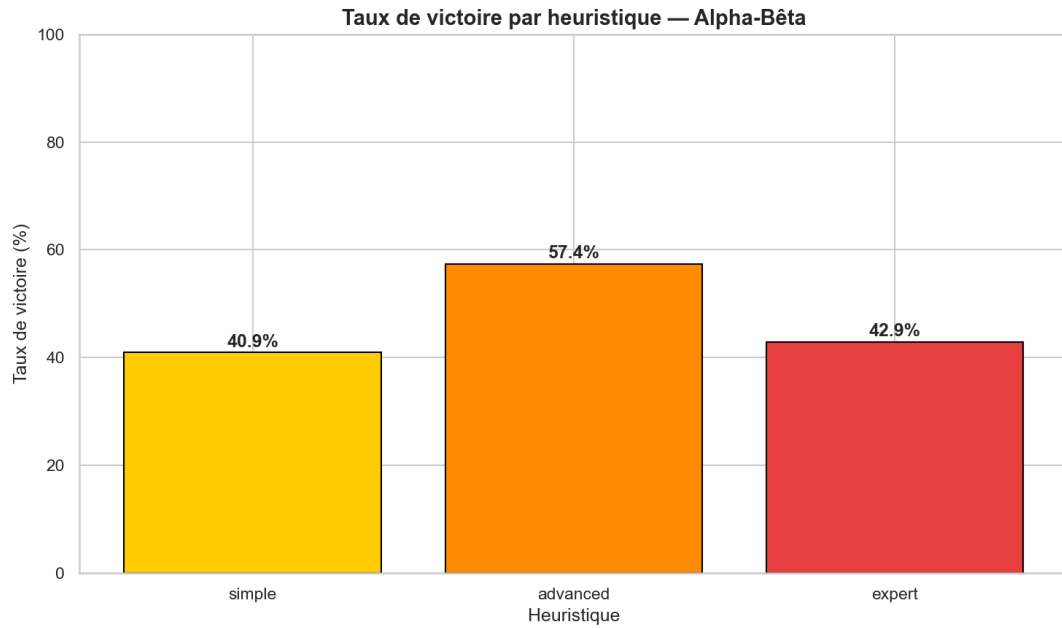


FIGURE 11 – Taux de victoire par heuristique — Alpha-Bêta

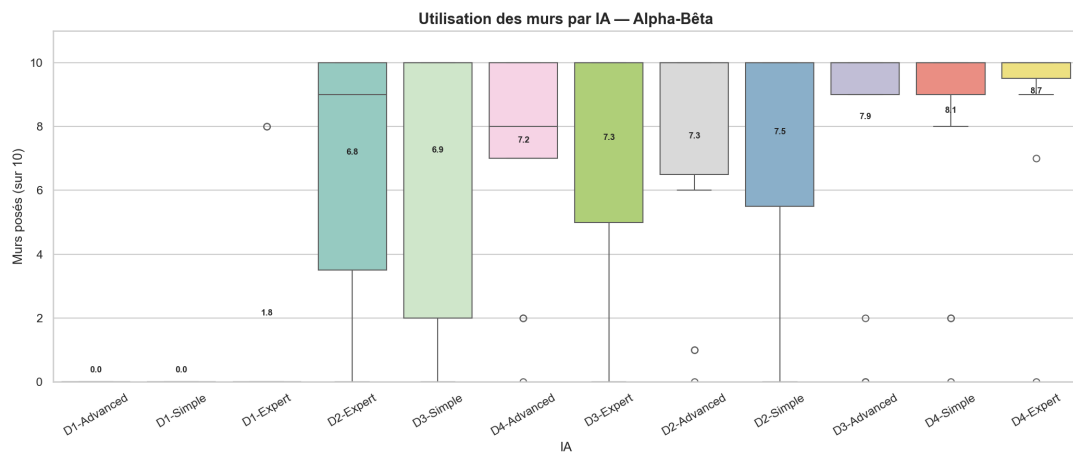


FIGURE 12 – Utilisation des murs — Alpha-Bêta

4.5 Comparaison Minimax pur vs Alpha-Bêta

Le résultat le plus frappant : l'heuristique dominante s'inverse entre les deux tournois.

Avec le Minimax pur, l'expert domine (podium 100% expert). Tous les nœuds sont explorés, donc ses 5 composantes évaluent chaque position de manière plus fine.

Avec l'Alpha-Bêta, c'est l'advanced qui prend le dessus (podium 100% advanced). Ses 2 termes plus stables guident mieux l'élagage, alors que les scores dispersés de l'expert rendent les coupures moins fiables.

TABLE 5 – Comparaison des trois tournois

	Minimax + random	Minimax pur	Alpha-Bêta
Durée totale	~2h	~2h	~20min
Heuristique dominante	expert	variée	advanced
Champion	D2-Expert	D4-Advanced	D4-Advanced
Tie-breaking	oui (0.2)	non	non
Hiérarchie profondeur	perturbée	partielle	respectée

Le tournoi Alpha-Bêta est environ 6 fois plus rapide que les tournois Minimax. Ça colle avec la théorie : $O(b^{d/2})$ contre $O(b^d)$, soit avec $b \approx 12$, environ $12^2 = 144$ nœuds par niveau à D4 pour l'élagage contre $12^4 = 20\,736$ pour le Minimax pur.

Le Minimax sans tie-breaking et l'Alpha-Bêta ont le même champion : D4-Advanced. Le Minimax avec random donne un classement incohérent (D2-Expert premier). Le tie-breaking perturbe surtout les profondeurs élevées, où D4 évalue plus de coups et tombe plus souvent sur des égalités.

L'heuristique advanced domine avec l'Alpha-Bêta (podium 100% advanced), mais pas avec le Minimax pur sans random où le classement est plus mélangé. Avec le Minimax pur, toutes les branches sont explorées, donc les différences entre heuristiques sont moins marquées.

5 Utilisation de l'IA générative

5.1 Outils utilisés

On a utilisé deux outils, Claude et Gemini :

- Claude via le navigateur, pour les discussions sur la conception, la réflexion sur l'architecture et les heuristiques, et l'aide au debug.
- Gemini via le navigateur, pour les recherches approfondies par rapport aux heuristiques, l'analyse des résultats obtenus, et la conception du rapport final.

5.2 Ce qu'on en a fait

- Conception : discussion du choix des heuristiques, des critères d'évaluation, de la structure du tournoi.
- Code : l'IA a généré le squelette du moteur de jeu, une première version du minimax, et l'optimisation.
- Débogage : debug par rapport au blocage des IA. Claude a suggéré d'ajouter une pénalité, il a aussi suggéré l'ajout du tie-breaking pour rendre le jeu moins déterministe, mais on n'a pas gardé ce dernier.
- Rapport : aide à la structuration et à la rédaction, relecture et correction du document.

5.3 Ce qui a marché et ce qui n'a pas marché

L'IA accélère le développement, mais le code n'est pas toujours correct du premier coup. Exemple : la première version du filtrage des murs ne vérifiait pas que l'IA ne se bloquait pas

elle-même. On ne l’a vu qu’en regardant une partie où l’IA posait un mur devant son propre pion.

L’anti-oscillation a demandé 3 ou 4 tentatives. Il fallait découpler la pénalité du score alpha-bêta, sinon l’élagage devenait incorrect. C’est le genre de subtilité que l’IA ne voit pas tout seul.

Le tie-breaking n’a pas énormément ajouté de valeur au projet : dans le tournoi, on se retrouvait avec une IA de profondeur 1 qui battait une IA de profondeur 4.

Chaque bout de code généré a été relu et testé. L’IA fait gagner du temps mais ne dispense pas de comprendre ce qu’on fait.

6 Difficultés rencontrées

6.1 Auto-blocage de l’IA

Le bug le plus long à résoudre. L’IA posait des murs sans vérifier si ça rallongeait son propre chemin. Résultat : elle se construisait un labyrinthe autour d’elle-même. La correction est dans `moves_optimization.py` : avant de garder un mur candidat, on simule sa pose et on vérifie avec un BFS que le gain de blocage dépasse le coût pour nous.

6.2 Oscillation entre positions

Les IA D1 faisaient des aller-retours entre deux cases. À profondeur 1, l’IA ne voit pas assez loin pour comprendre qu’elle tourne en rond. On a mis un historique de 6 positions et une pénalité de -50 pour les revisiter. Point important : cette pénalité ne s’applique qu’au choix final, pas dans la récursion alpha-bêta (sinon l’élagage est faussé).

6.3 Temps de calcul à profondeur 3

À D3, chaque coup prend environ 20 secondes. Pour un tournoi, c’est trop. Deux optimisations :

- `_fast_copy` : copie manuelle du board (attribut par attribut) au lieu de `copy.deepcopy`. Environ 50 fois plus rapide, parce que nos attributs sont des types immutables.
- La réduction du facteur de branchement (de 72 à 10–15 coups) décrite en section 3.

Avec les deux, le tournoi tourne en 15 minutes environ.

6.4 Gestion des sauts

Plus compliqué que prévu. Il faut gérer le saut direct (par-dessus l’adversaire), le diagonal (quand un mur bloque derrière), le cas du bord (pas de saut hors de la grille), et les combinaisons. On a écrit des tests dédiés pour chaque cas.

6.5 Pose de murs lors d’un match entre deux IA

Au début du développement, quand une IA jouait contre une autre, elles ne posaient aucun mur. Les deux IA allaient tout droit, et ce pour toutes les profondeurs et heuristiques implémentées. Pour régler ce problème, on a dû ajouter des récompenses supplémentaires afin qu’une IA pose des murs.